

Сетевое взаимодействие в K8s

DNS, типы Services, ingress

Кирилл Касаткин
DevOps-инженер, Renue

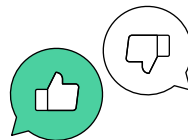


Проверка связи



Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти



Поставьте в чат:

- ☒ + если меня видно и слышно
- ☐ — если нет

Рекомендации

→ Если смотрите с компьютера

- Используйте браузеры **Google Chrome** или **Microsoft Edge**
- Если есть проблемы с изображением или звуком, обновите страницу — **F5**

→ Если смотрите с мобильного телефона или планшета

- Перейдите с мобильного интернет-соединения на **Wi-Fi**
- Если есть проблемы с изображением или звуком, перезапустите приложение **МТС-Линк** на телефоне

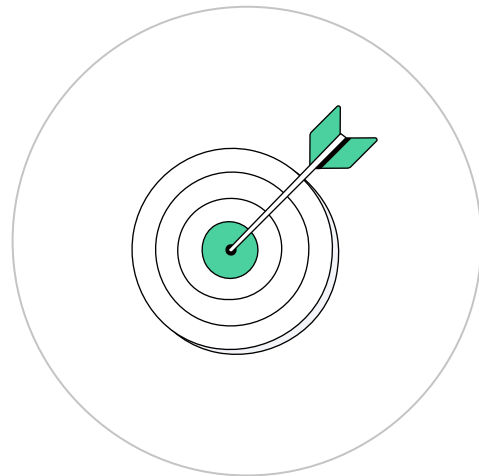
Кирилл Касаткин

DevOps-инженер, Renu



Цели занятия

- Узнать:
 - какие DNS-записи создаются внутри кластера
 - какие типы Service бывают и для чего они нужны
 - что такое ingress и ingress controller
- Познакомиться с примерами манифестов различных сервисов
- Понять, каким образом можно получить доступ к приложениям снаружи кластера
- Разобрать примеры манифестов объектов K8s



План занятия

- 1 DNS для подов и сервисов
- 2 Типы Service
- 3 ClusterIP, NodePort
- 4 LoadBalancer
- 5 Ingress
- 6 Ingress Controller
- 7 Итоги



Вспоминаем прошное занятие

Вопрос: что такое Service?



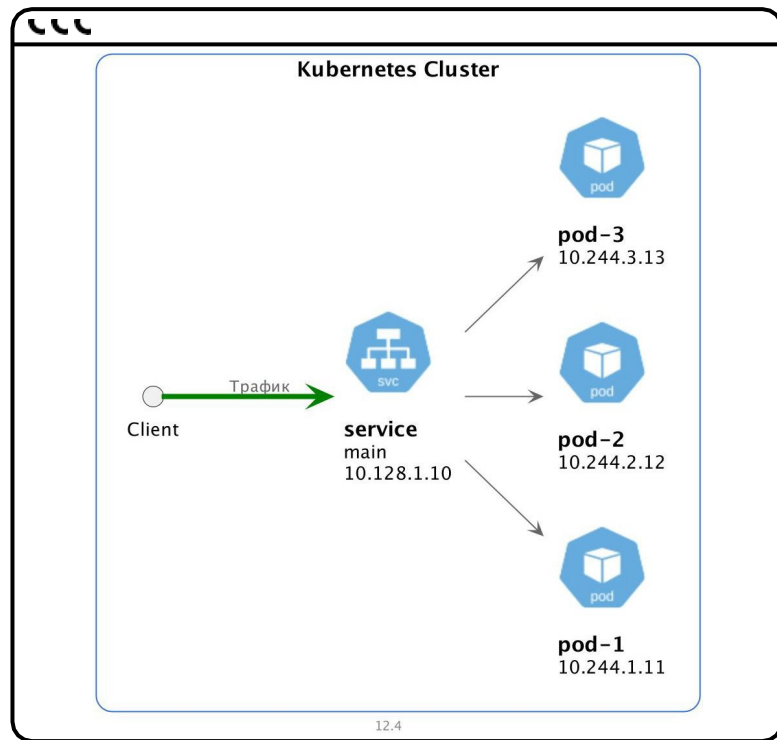
Вспоминаем прошлые занятия

Вопрос: что такое Service?

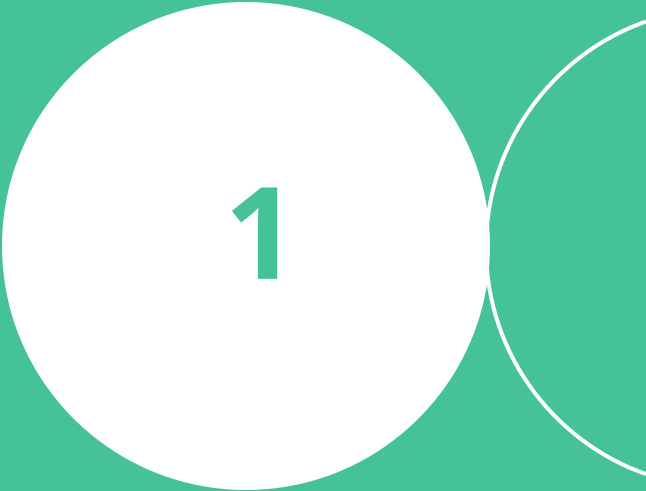
Ответ: ресурс K8s, который предоставляет возможность доступа к приложению, состоящему из множества подов на L4-уровне (protocol:port)



Вспоминаем прошлое занятие



DNS для подов и сервисов



1

DNS для подов и сервисов

K8s создаёт DNS-записи для сервисов и подов.

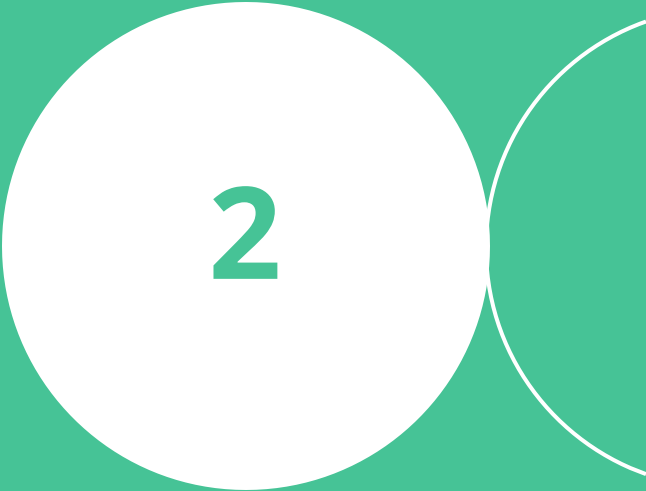
Сервисы и поды внутри Namespace могут быть доступны по короткому имени

Сервисы и поды имеют записи следующего формата:

```
service-name.namespace-name.svc.cluster-domain.example  
pod-ip-address.namespace-name.pod.cluster-domain.example
```

Типы Service

ClusterIP, NodePort, LoadBalancer



2

Типы Service

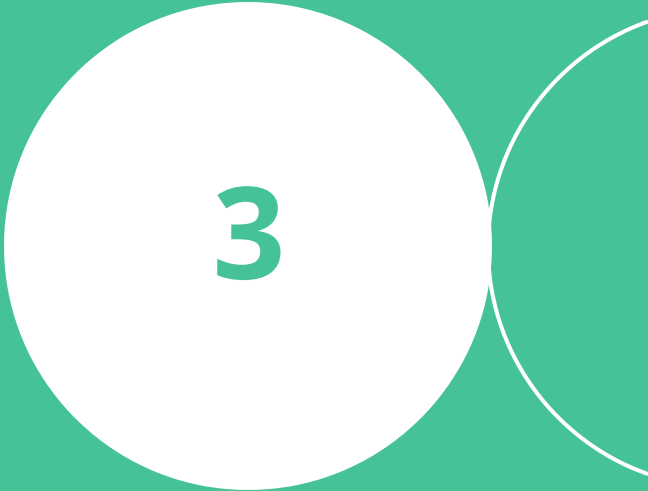
Иногда требуется обеспечить доступ к приложению извне кластера K8s.

Пример: внешний мир должен иметь доступ к frontend вашего приложения.
В таком случае требуется использовать другие типы Service

Тип сервиса определяет его поведение

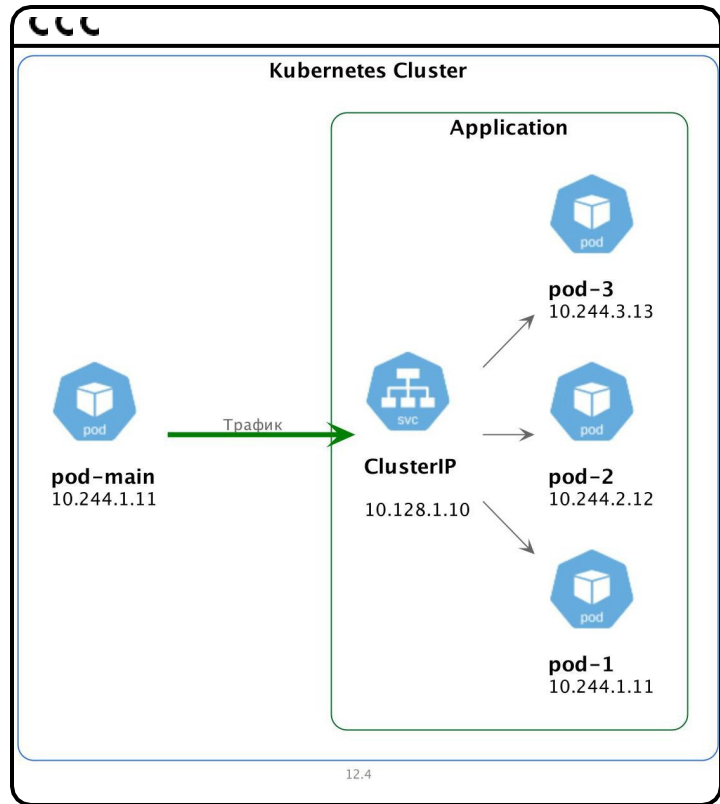
- 1 ClusterIP
- 2 NodePort
- 3 LoadBalancer

ClusterIP, NodePort

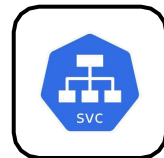


3

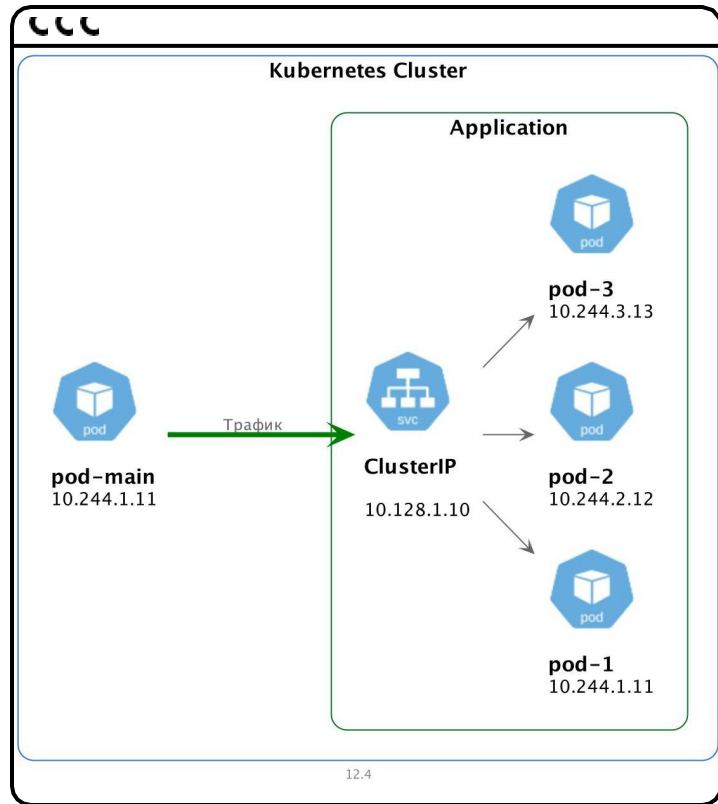
Типы Service. ClusterIP



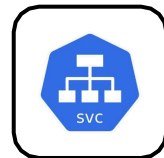
- Выставляет сервис внутри кластера на внутреннем IP-адресе



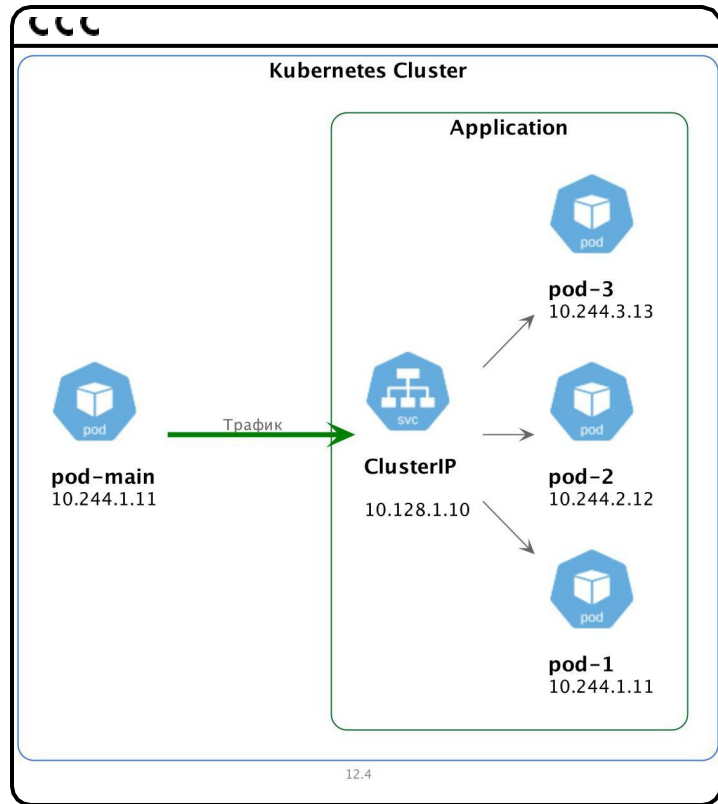
Типы Service. ClusterIP



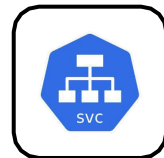
- Выставляет сервис внутри кластера на внутреннем IP-адресе
- Такой сервис доступен **только** внутри кластера



Типы Service. ClusterIP



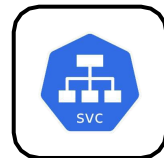
- Выставляет сервис внутри кластера на внутреннем IP-адресе
- Такой сервис доступен **только** внутри кластера
- Является типом по умолчанию



Пример конфигурации

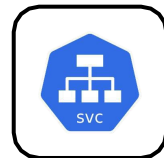
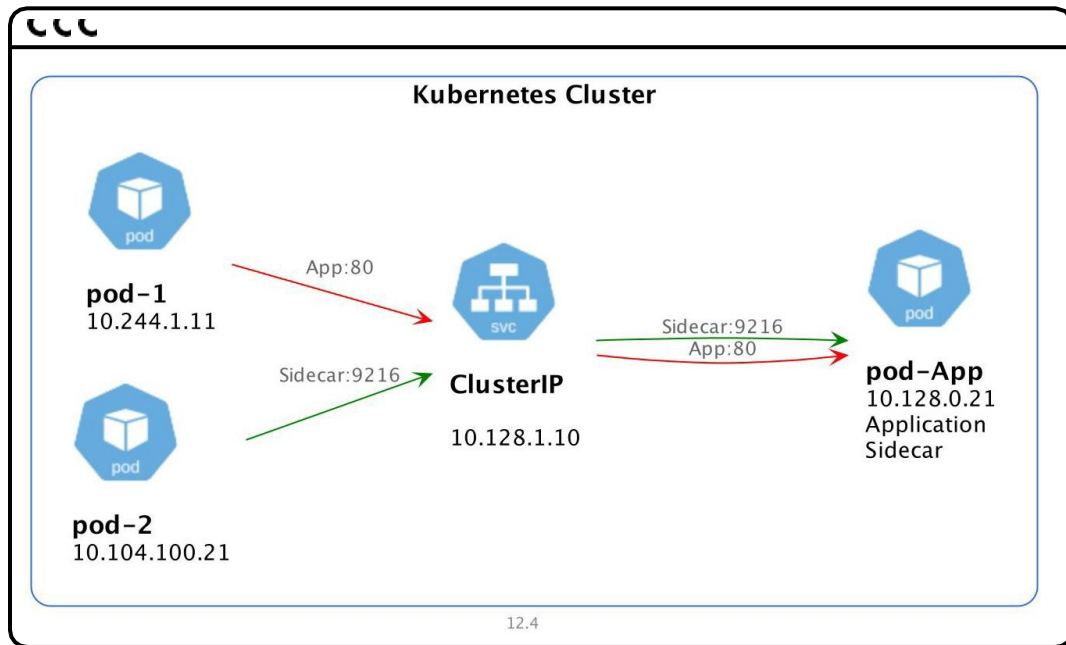
Пример конфигурации Service типа ClusterIP

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
  namespace: default
spec:
  ports:
    - name: web
      port: 80
  selector:
    app: nginx
  type: ClusterIP
```



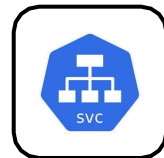
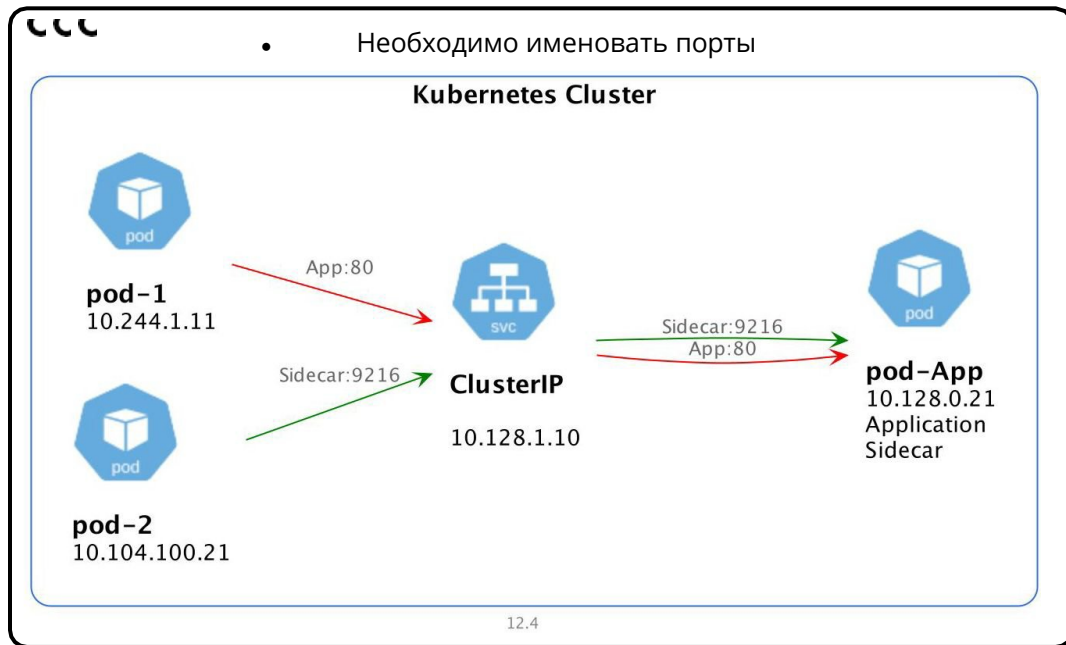
Типы Service. ClusterIP MultiPort

ClusterIP MultiPort используется тогда, когда необходимо получить доступ к нескольким портам пода



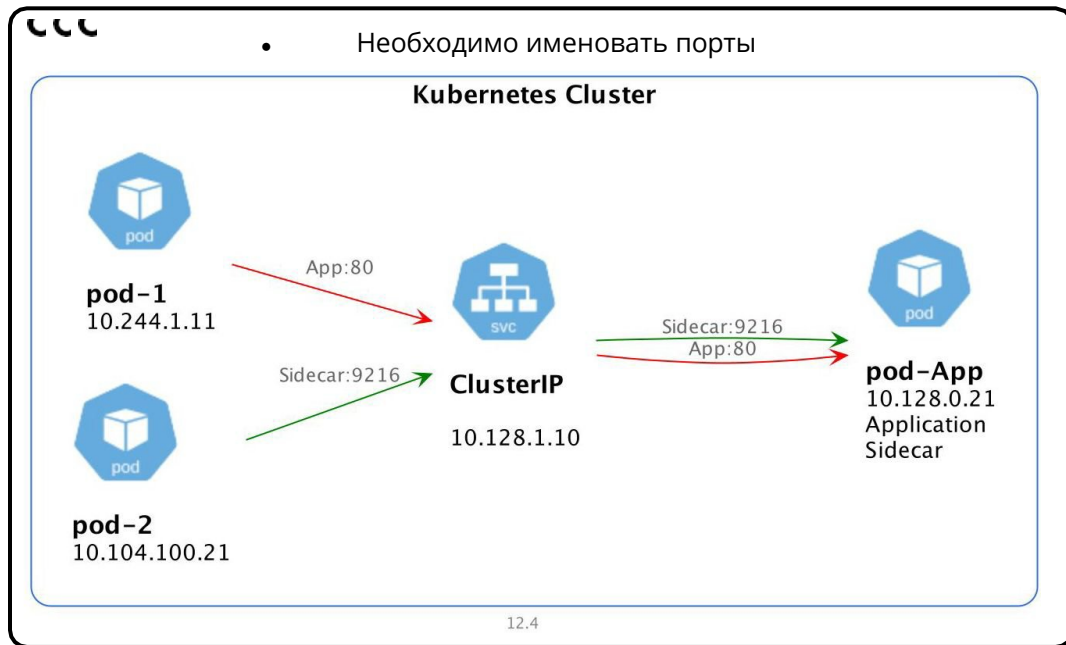
Типы Service. ClusterIP MultiPort

ClusterIP MultiPort используется тогда, когда необходимо получить доступ к нескольким портам пода

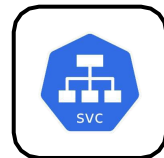


Типы Service. ClusterIP MultiPort

ClusterIP MultiPort используется тогда, когда необходимо получить доступ к нескольким портам пода



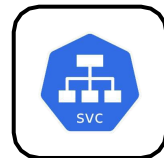
- Обычно используется для доступа к различным контейнерам (sidecar)



Пример конфигурации

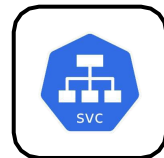
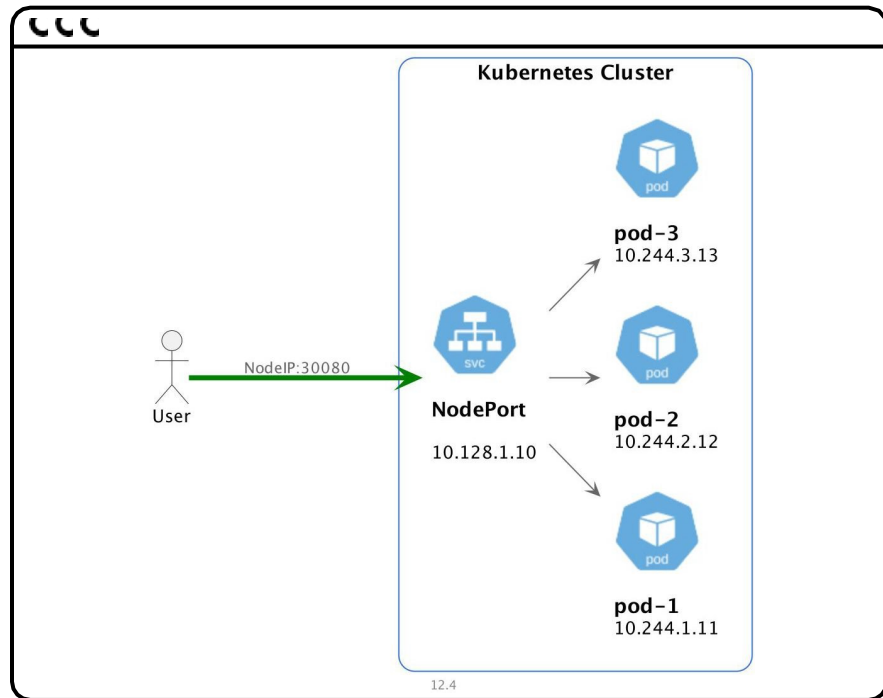
Пример конфигурации Service типа ClusterIP MultiPort

```
apiVersion: v1
kind: Service
metadata:
  name: my-svc
spec:
  selector:
    app: MyApp
  ports:
    - name: web
      protocol: TCP
      port: 80
      targetPort: 8080
    - name: db
      protocol: TCP
      port: 9216
      targetPort: 19216
```



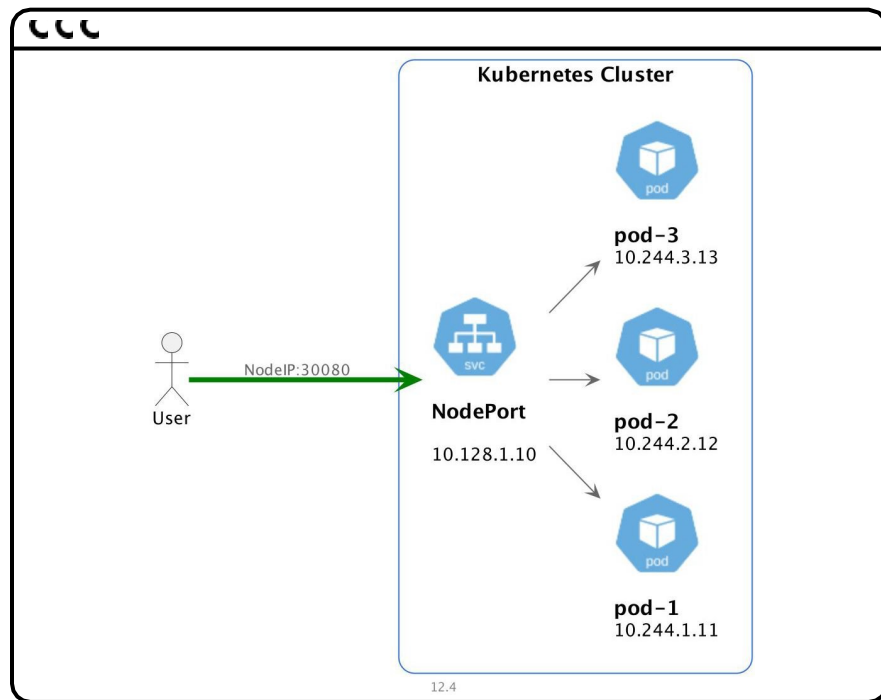
Типы Service. NodePort

Делает доступным сервис на статическом порту ноды, где запущен

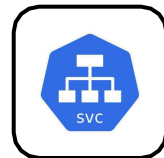


Типы Service. NodePort

Делает доступным сервис на статическом порту ноды, где запущен

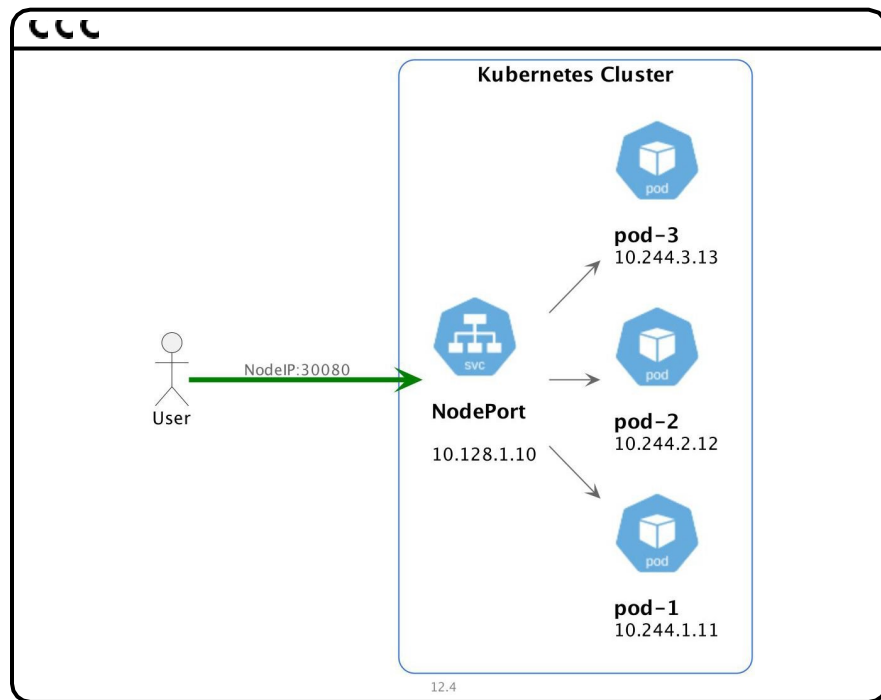


- Можно получить доступ к сервису снаружи кластера по IP-адресу ноды. Этот NodePort открыт на **каждом узле** даже в тех случаях, когда ни один под на узле не запущен

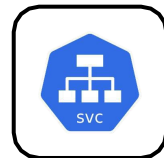


Типы Service. NodePort

Делает доступным сервис на статическом порту ноды, где запущен

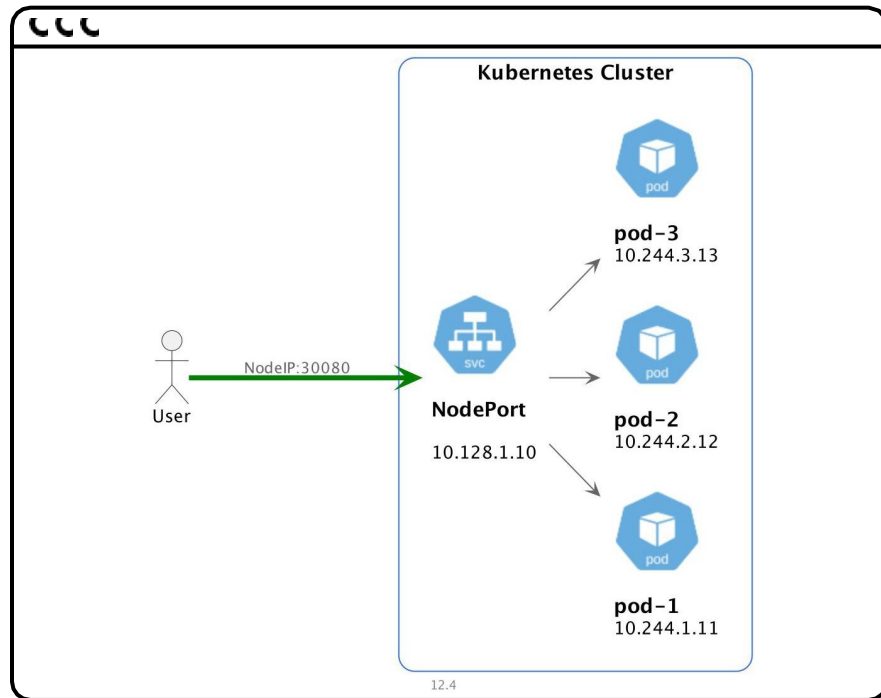


- Можно получить доступ к сервису снаружи кластера по IP-адресу ноды. Этот NodePort открыт на **каждом узле** даже в тех случаях, когда ни один под на узле не запущен
- Номер порта для сервиса NodePort должен быть в диапазоне 30 000–32 767

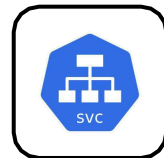


Типы Service. NodePort

Делает доступным сервис на статическом порту ноды, где запущен



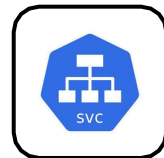
- Можно получить доступ к сервису снаружи кластера по IP-адресу ноды. Этот NodePort открыт на **каждом узле** даже в тех случаях, когда ни один под на узле не запущен
- Номер порта для сервиса NodePort должен быть в диапазоне 30 000–32 767
- При создании сервиса NodePort, будет автоматически создан ещё и сервис ClusterIP, к которому обращается сервис NodePort



Пример конфигурации

Пример конфигурации Service типа NodePort

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
  namespace: default
spec:
  ports:
    - name: web port:
      80 protocol:
        TCP nodePort:
          30080
  selector:
    app: nginx
  type: NodePort
```



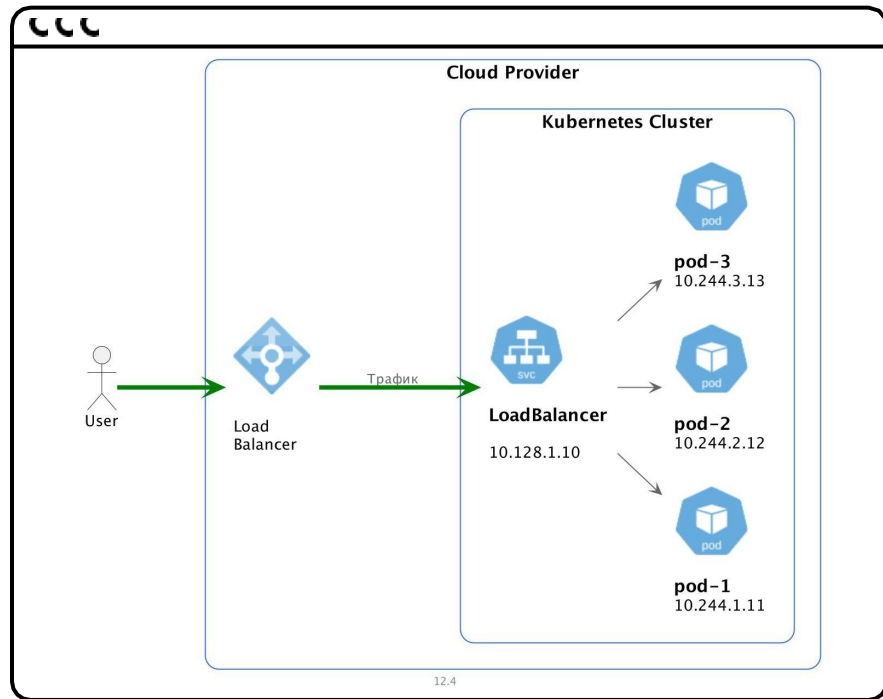
LoadBalancer



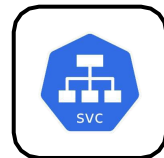
4

Типы Service. LoadBalancer

Делает доступным сервис снаружи кластера с помощью облачного балансировщика нагрузки

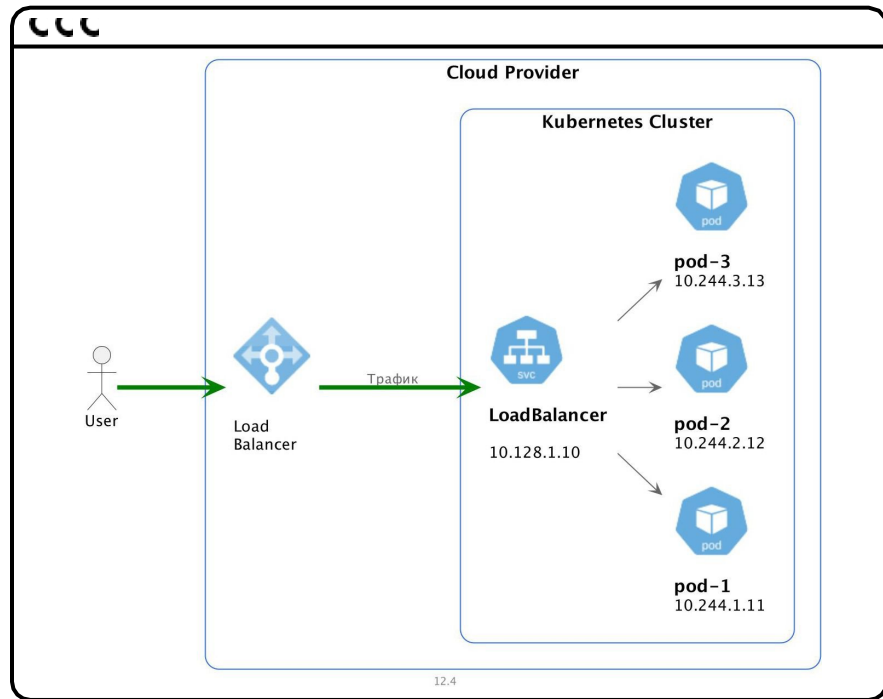


Источник: архив автора

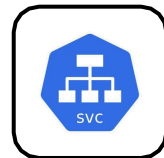


Типы Service. LoadBalancer

Делает доступным сервис снаружи кластера с помощью облачного балансировщика нагрузки

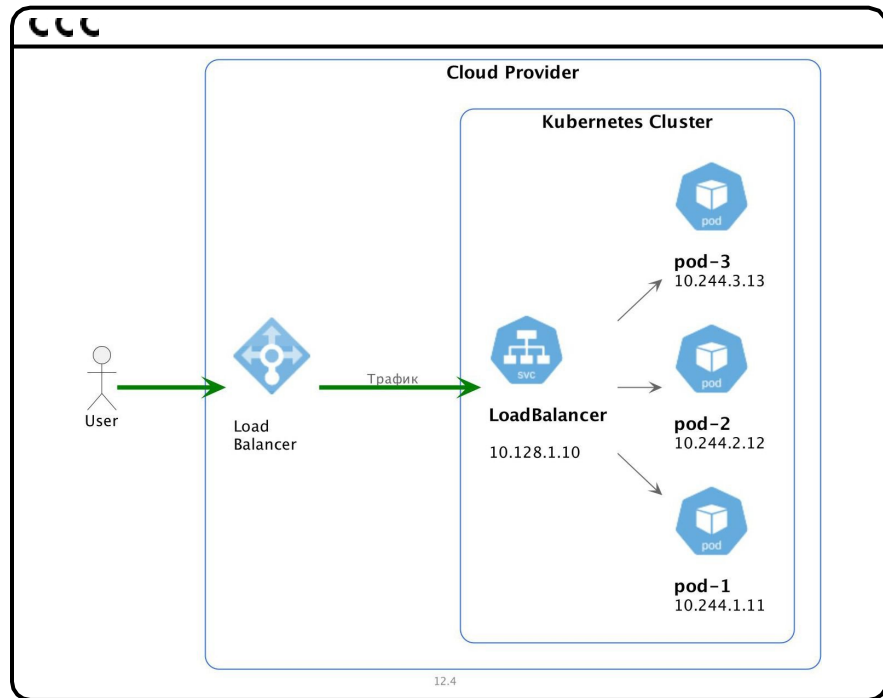


- Автоматически создаются сервисы NodePort и ClusterIP, к которым обращается сервис LoadBalancer

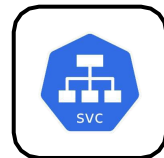


Типы Service. LoadBalancer

Делает доступным сервис снаружи кластера с помощью облачного балансировщика нагрузки



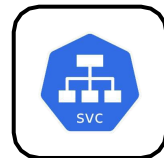
- Автоматически создаются сервисы NodePort и ClusterIP, к которым обращается сервис LoadBalancer
- Этот тип сервиса работает только в облачных провайдерах (используется Cloud Controller K8s). В противном случае такой сервис зависнет в стадии Pending



Пример конфигурации

Пример конфигурации Service типа LoadBalancer

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
  namespace: default
spec:
  ports:
    - name: web
      port: 80
      protocol: TCP
      targetPort: 9376
  selector:
    app: nginx
  clusterIP:
    10.0.171.239
  type:
    LoadBalancer
```

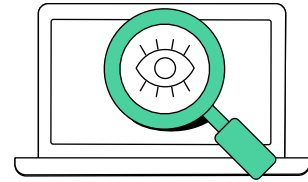




Ваши вопросы?

Демонстрация работы

Работа с различными типами Service, связь с DNS



Закрепляем

Вопрос: что такое Service?



Закрепляем

Вопрос: что такое Service?

Ответ: ресурс K8s, который предоставляет возможность доступа к приложению, состоящему из множества подов (дает возможность получить доступ извне)



Закрепляем

Вопрос: какой тип Service лучше использовать для получения доступа к приложению извне?



Закрепляем

Вопрос: какой тип Service лучше использовать для получения доступа к приложению извне?

Ответ: Service с типом NodePort или LoadBalancer подойдет для этого лучше всего



Ingress



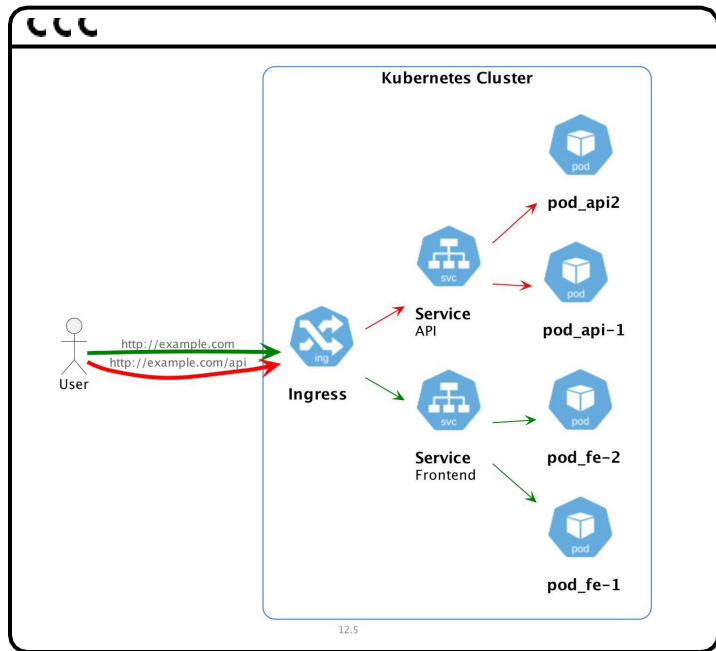
5



**Ingress — объект K8s,
который обеспечивает
внешний доступ
к сервисам на 7 уровне
(HTTP/HTTPS)**

Ingress маршрутизирует трафик

Ingress консолидирует правила маршрутизации трафика в одном месте, что позволяет использовать несколько сервисов под одним IP-адресом

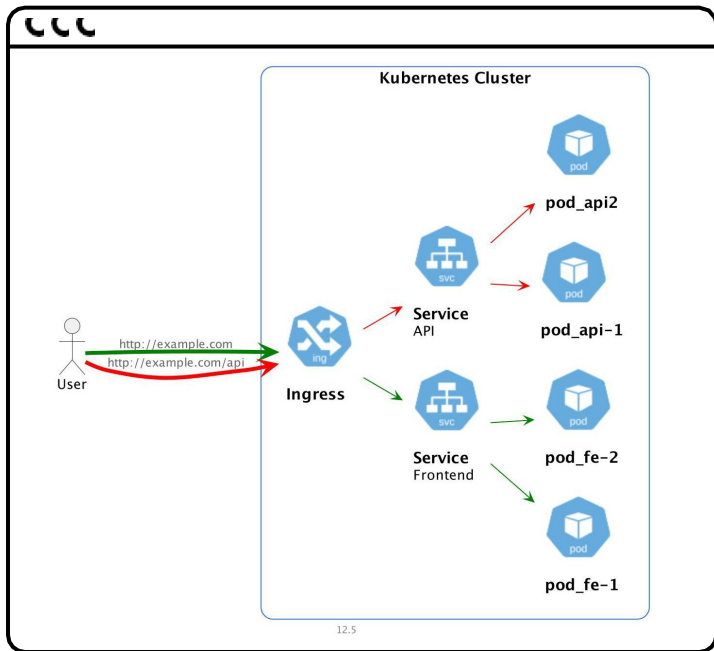


Источник: архив автора



Ingress маршрутизирует трафик

Ingress консолидирует правила маршрутизации трафика в одном месте, что позволяет использовать несколько сервисов под одним IP-адресом

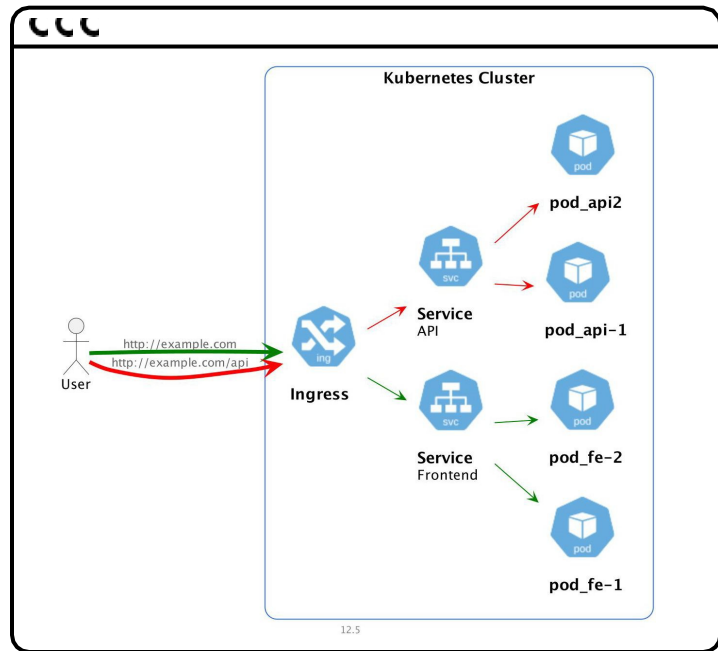


- Ingress определяет правила маршрутизации трафика — Rule



Ingress маршрутизирует трафик

Ingress консолидирует правила маршрутизации трафика в одном месте, что позволяет использовать несколько сервисов под одним IP-адресом

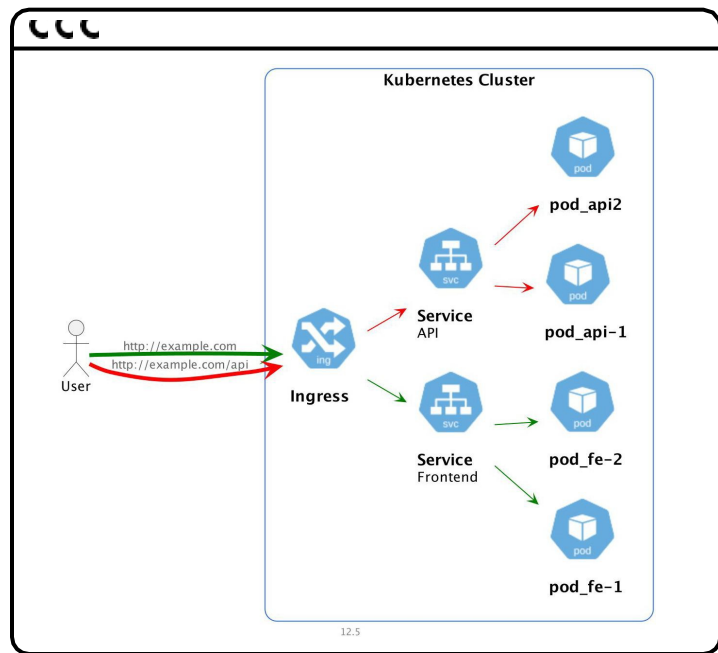


- Ingress определяет правила маршрутизации трафика — Rule
- Каждое Rule определяет Path со своим бэкэндом, указывающим на Service. Запросы определёнными Path будут направлены в соответствующие Service



Ingress маршрутизирует трафик

Ingress консолидирует правила маршрутизации трафика в одном месте, что позволяет использовать несколько сервисов под одним IP-адресом



- Ingress определяет правила маршрутизации трафика — Rule
- Каждое Rule определяет Path со своим бэкэндом, указывающим на Service. Запросы определёнными Path будут направлены в соответствующие Service
- Можно ссылаться на имя порта вместо номера, если он указан в настройке сервиса



Пример конфигурации

Примеры конфигураций Ingress HTTP

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  rules:
  - host: my-app.com
    http:
      paths:
      - path: /
        backend:
          service:
            name: my-app-service
            port:
              number: 80
      - path: /api
        backend:
          service:
            name: my-app-api
            port:
              number: 80
```

Источник: архив автора



Пример конфигурации

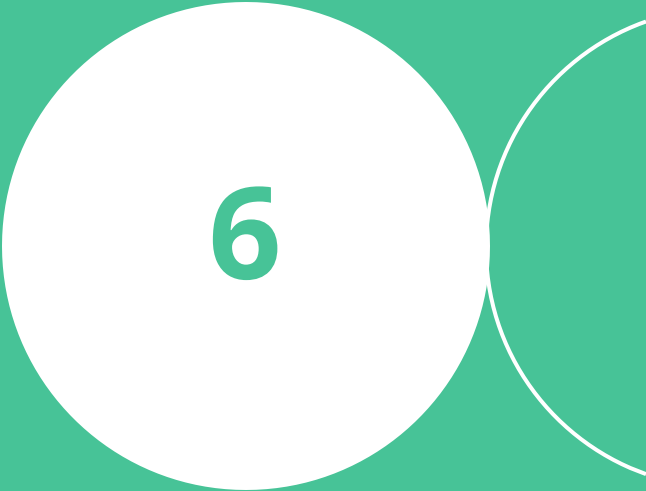
Примеры конфигураций Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  rules:
  - host: my-app.com
    http:
      paths:
      - path: /
        backend:
          service:
            name: my-app-service
            port:
              number: 80
      - path: /api
        backend:
          service:
            name: my-app-api
            port:
              number: 80
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  rules:
  - host: my-app.com
    http:
      paths:
      - backend:
          service:
            name: my-app-service
            port:
              number: 80
      - host: api.my-app.com
        http:
          paths:
          - backend:
              service:
                name: my-app-api
                port:
                  number: 80
```



Ingress Controller

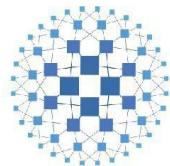


6



**Ingress controller —
под (набор подов)
с запущенным приложением-
контроллером, выполняющим
правила Ingress**

Ingress controller бывают разные



HAPROXY



CONTOUR



Kong

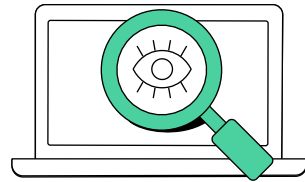




Ваши вопросы?

Демонстрация работы

Конфигурация Ingress с несколькими правилами маршрутизации, Nginx Ingress Controller



Закрепляем

Вопрос: какой путь проходит запрос извне, чтобы попасть к вашему приложению?



Закрепляем

Вопрос: какой путь проходит запрос извне, чтобы попасть к вашему приложению?

Ответ: Ingress (ingress controller) → Service (NodePort) → Service (ClusterIP) → App



Итоги занятия

7

Итоги занятия

Сегодня мы:

- Узнали, как формируются DNS-имена у подов и сервисов
- Разобрались, какие типы сервисов бывают и где применяется каждый из них
- Узнали, что такое Ingress и Ingress controller.
Обсудили, какие виды Ingress controller существуют
- Разобрались, как происходит маршрутизация трафика
- Рассмотрели примеры манифестов объектов K8s
- Поняли, как получить доступ к приложениям снаружи кластера
- Подключились к кластеру и посмотрели в работе изученные объекты



Сетевое взаимодействие в K8s

DNS, типы Services, ingress

Кирилл Касаткин
DevOps-инженер, Renue

